.NET Programmierung 2026 – Aufgabe 03

Worum geht es im nächsten Schritt?

In Aufgabe 02 haben Sie Ihre Simulatoren zu einem verteilten Teamsystem verbunden. TimeSim steuert die gemeinsame virtuelle Zeit, WeatherSim liefert konsistente Wetterwerte, und die Gerätesimulatoren stellen ihre Zustände über Modbus TCP bereit.

Im nächsten Schritt übertragen Sie die über Modbus gelesenen Gerätedaten an einen Nachrichtenbroker. RabbitMQ läuft lokal unter Docker. Ein Viewer empfängt die Nachrichten und macht den Datenfluss sichtbar. Damit entsteht die Grundlage für die spätere Speicherung in InfluxDB.

Gleichzeitig entwickeln Sie Ihre Arbeitsweise mit KI weiter: Anforderungen und Umsetzung planen, Projektanweisungen gezielt pflegen und wiederkehrende Abläufe als Skills beschreiben. Sie steuern die Entwicklung und prüfen die Ergebnisse selbst.

Aufgabe 03: Von Modbus zu RabbitMQ

Erweitern Sie Ihr bestehendes Teamprojekt um den folgenden durchgängigen Datenweg:

```
Gerätesimulator mit Modbus-Server
    → Modbus–RabbitMQ-Gateway (Modbus-Client und Publisher)
    → RabbitMQ unter Docker
    → Nachrichtenviewer
```

Das **Modbus–RabbitMQ-Gateway** liest als Modbus-TCP-Client die Gerätedaten, dekodiert sie gemäß Registerplan und veröffentlicht sie über AMQP oder MQTT bei RabbitMQ. Es vermittelt damit zwischen unterschiedlichen Protokollen und Datenmodellen.

Die bestehenden Simulatoren, die gemeinsame virtuelle Zeit und die Modbus-Diagnose bleiben funktionsfähig. Wie Sie die neuen Aufgaben auf Anwendungen und gemeinsame Assemblies verteilen, entscheiden und begründen Sie im Team.

1. Verantwortlichkeiten und Pflichtumfang

Jede Person übernimmt die Hauptverantwortung für zwei Simulatoren. Dokumentieren Sie die konkrete Zuordnung im zentralen `README.md`:

Teamgröße	Simulatoren	Verantwortung
Zweierteam	TimeSim, WeatherSim, PV sowie vereinbarter Batteriespeicher oder Dual-Port-Charger	zwei Simulatoren pro Person
Dreierteam	TimeSim, WeatherSim, PV, Batteriespeicher, Dual-Port-Charger und Wind/Industrie	zwei Simulatoren pro Person

Binden Sie **alle verpflichtenden Gerätesimulatoren** Ihres Teams an RabbitMQ an. Die RabbitMQ-Anbindung von **TimeSim und WeatherSim ist optional**; beide Programme bleiben als Bestandteile des Simulationssystems verpflichtend. Zwei Simulatoren in Verantwortung bedeuten daher nicht zwingend zwei RabbitMQ-Anbindungen pro Person.

Ordnen Sie auch die neuen Module und die Integration nachvollziehbar zu. Alle Teammitglieder müssen den gesamten Datenweg und den eigenen Beitrag erklären können.

2. Anforderungen und Umsetzung mit KI planen

Erstellen Sie vor der Implementierung eine kurze SRS (**Software Requirements Specification**) für diese Ausbaustufe, beispielsweise unter `docs/SRS-LB03.md`. Beschreiben Sie darin:

- Ziel und Umfang der Erweiterung,
- den Datenweg und die beteiligten Anwendungen,
- die funktionalen Anforderungen an Gateway und Viewer,
- Nachrichtenformat, Konfiguration und Verhalten bei Verbindungsfehlern,
- überprüfbare Akzeptanzkriterien.

Nutzen Sie einen Coding-Agenten, um Anforderungen und Lösungsvarianten zu diskutieren. Prüfen und überarbeiten Sie dessen Vorschläge, bevor Sie die Implementierung beauftragen. Zerlegen Sie den Plan in kleine, überprüfbare Schritte. Eine knappe, projektspezifische Spezifikation genügt; eine umfangreiche formale Dokumentation ist nicht erforderlich.

3. RabbitMQ unter Docker betreiben

Installieren Sie eine für Ihr Betriebssystem geeignete Docker-Umgebung und nehmen Sie RabbitMQ lokal in Betrieb. Docker Desktop beziehungsweise Portainer können Sie als Verwaltungsoberfläche verwenden. Für einen Portainer-basierten Aufbau benötigen Sie ebenfalls eine funktionsfähige Docker-Umgebung.

Übernehmen Sie eine zum Projekt passende Compose-Datei und gegebenenfalls weitere Broker-Konfigurationen in Ihr Repository. Dokumentieren Sie:

- verwendetes RabbitMQ-Image mit Versionsangabe,
- benötigte Ports und gegebenenfalls Plugins,
- Konfiguration und Einrichtung der lokalen Zugangsdaten,
- Start, Statuskontrolle und kontrolliertes Stoppen des Brokers.

Die Konfiguration muss zu Ihrem verwendeten Protokoll passen. Verwenden Sie AMQP oder MQTT und begründen Sie Ihre Wahl kurz. Für MQTT konfigurieren Sie das entsprechende RabbitMQ-Plugin. Orientieren Sie sich an der im Unterricht verwendeten Demo und prüfen Sie die übernommenen Einstellungen. Nutzen Sie bei Bedarf die offiziellen Dokumentationen zu Docker Compose und zum RabbitMQ-MQTT-Plugin.

Versionieren Sie keine tatsächlichen Zugangsdaten. Stellen Sie eine `.env.example` oder eine gleichwertige Beispielkonfiguration mit Platzhaltern bereit und dokumentieren Sie die lokale Einrichtung. Die selbst entwickelten Anwendungen dürfen weiterhin in eigenen Terminals laufen; ihre Containerisierung ist nicht erforderlich.

4. Modbus–RabbitMQ-Gateway implementieren

Erweitern Sie den Modbus-Client aus Aufgabe 02 zum Modbus–RabbitMQ-Gateway oder erstellen Sie auf dessen Grundlage eine eigene Gateway-Anwendung. Lesen Sie die aktuellen Werte aller verpflichtenden Gerätesimulatoren über deren Modbus-Schnittstellen und veröffentlichen Sie diese an RabbitMQ.

Das Gateway muss:

- Geräte über konfigurierbare Kennungen, Hosts und Ports anbinden,
- den gemeinsamen Registerkopf und die gerätespezifischen Werte gemäß Registerplan dekodieren,
- Abfrage- und Veröffentlichungsintervall konfigurierbar machen,
- den tatsächlich gelesenen Simulationszeitpunkt beziehungsweise dessen dokumentierte Zuordnung verwenden,
- Verbindungs- und Dekodierungsfehler sichtbar melden,
- kontrolliert beendet werden können.

Die Gerätedaten werden über Modbus gelesen. Ersetzen Sie diesen Datenweg nicht durch direkten Zugriff auf interne Modellobjekte oder frei erfundene Messwerte. Der Diagnose-Client aus Aufgabe 02 muss weiterhin verwendet werden können.

Eine Nachricht beschreibt ein über `DeviceId` eindeutig identifiziertes Gerät zu einem veröffentlichten Simulationszeitpunkt. Sie darf mehrere Messwerte dieses Geräts enthalten. Dokumentieren Sie mindestens:

- Gerätekennung `DeviceId`,
- virtuelle Simulationszeit mit eindeutiger Zeitdarstellung,
- Sequenznummer und deren Bedeutung,
- Qualitätsstatus der Daten,
- Messwertname, Wert und Einheit für die übertragenen Messwerte.

Legen Sie gemeinsam ein verständliches Nachrichtenformat und die benötigten Topics beziehungsweise Exchanges, Queues und Bindings fest. Dokumentieren Sie die für Ihr Protokoll relevanten Angaben sowie mindestens eine Beispielnachricht. Eine `TeamId` ist nicht erforderlich.

Die virtuelle Zeit der Messwerte und die reale Abfragezeit sind unterschiedliche Größen. Stellen Sie sicher, dass diese nicht verwechselt werden. Eine atomare Lese-Garantie über mehrere Modbus-Register ist auch in dieser Aufgabe nicht verlangt; die zeitliche Zuordnung und bekannte Einschränkungen müssen nachvollziehbar sein.

5. Nachrichten empfangen und sichtbar machen

Richten Sie einen zum gewählten Protokoll passenden Viewer ein. Sie können die bereitgestellte DemoApp zu einem Empfänger weiterentwickeln. Bei MQTT ist alternativ ein geeignetes Werkzeug wie MQTT Explorer möglich; ein selbst entwickelter Viewer ist dann nicht zusätzlich erforderlich.

Der Viewer muss die Nachrichten aller verpflichtend angebundenen Geräte empfangen und mindestens Gerätekennung, Simulationszeit und übertragene Messwerte mit ihren Einheiten nachvollziehbar anzeigen. Die Zuordnung von Sequenznummer und Qualitätsstatus muss erkennbar sein, beispielsweise im angezeigten Nachrichteninhalt. Eine übersichtliche Konsolenausgabe genügt.

Dokumentieren Sie Verbindungseinstellungen und Abonnements beziehungsweise Routing so, dass der Empfang reproduzierbar eingerichtet werden kann. Der Viewer empfängt die Nachrichten über RabbitMQ; er liest nicht selbst die Modbus-Register.

6. Projektanweisungen pro Team und eigener Skill pro Person

Erstellen beziehungsweise überarbeiten Sie als Team eine **`AGENTS.md`** **oder** **`CLAUDE.md`**, passend zum verwendeten Coding-Agenten, für den aktuellen Projektstand. Sie enthält insbesondere die Solution-Struktur, Zuständigkeiten, Befehle zum Bauen, Testen und Starten sowie Regeln für Konfiguration, Geheimnisse und generierte Dateien. Halten Sie die Anweisungen kurz und erklären Sie im Team, warum die einzelnen Regeln für Ihr Projekt sinnvoll sind.

Jede Person implementiert und verwendet mindestens **einen eigenen Skill**, der einen sinnvollen, wiederkehrenden Arbeitsablauf im konkreten Teamprojekt unterstützt. Damit entstehen im Zweierteam mindestens zwei, im Dreierteam mindestens drei eigene Skills. Stimmen Sie die Aufgaben der Skills im Team ab. Dokumentieren Sie je Skill die verantwortliche Person, Zweck, Aufruf, benötigten Kontext, Arbeitsschritte und die Grenzen erlaubter Aktionen. Jede Person muss ihren Skill, seinen Nutzen und seine Wirkung erklären können.

Mögliche, auch auf andere Projekte übertragbare Anwendungsfälle sind:

- **Projektstand übergeben:** erledigte Arbeit, Entscheidungen, Prüfstand, offene Punkte und nächsten Schritt sichern, damit eine neue Sitzung nachvollziehbar fortsetzen kann.
- **Entwicklungsjournal ergänzen:** Änderungen und Prüfungen als Grundlage lesen, den nächsten persönlichen Eintrag mit der betreffenden Person abstimmen und erst nach deren Bestätigung in ihre Journaldatei eintragen. Der Skill darf persönliche Leistungen und KI-Erfahrungen nicht aus der Git-Historie erfinden.
- **Aufgabenblatt mit Umsetzung abgleichen:** das bereitgestellte Aufgaben-PDF lesen, Pflichtanforderungen und optionale Erweiterungen unterscheiden und für jede Anforderung Belege in Code, Konfiguration, Dokumentation oder Tests suchen. Das Ergebnis unterscheidet „erfüllt“, „teilweise erfüllt“, „nicht erfüllt“ und „nicht nachgewiesen“; bloß vorhandener Code beweist noch keinen erfolgreichen Lauf.
- **Arbeitsstand committen und pushen:** Diff und Prüfstand kontrollieren, die beabsichtigten Projektdateien gezielt committen und an das ausdrücklich bestimmte Remote und den Branch pushen. Zugangsdaten, IDE-Zustände sowie `bin/` und `obj/` bleiben ausgeschlossen. Keine fremden Änderungen überschreiben oder Force-Pushes ausführen; bei Konflikten oder fehlgeschlagenen Prüfungen den Grund berichten.
- **Abgabestand im Remote prüfen:** den vorgesehenen Abgabe-Tag und Commit im GitHub-/GitLab-Repository prüfen und diesen Stand mit den Abgabeanforderungen vergleichen. Lokale, noch nicht gepushte Dateien zählen nicht als abgegeben. Fehlende Dateien, Nachweise oder nicht prüfbare Anforderungen im Bericht ausweisen.
- **C#-Konventionen prüfen:** die vereinbarten Projektregeln und ausgewählte offizielle C#-Richtlinien erklären beziehungsweise auf ausgewählte Dateien anwenden und Verstöße mit Fundstellen berichten. Dauerhaft geltende Regeln gehören in die Projektanweisungen beziehungsweise `.editorconfig`; der Skill beschreibt den Ablauf ihrer Anwendung und Prüfung.
- **Dokumentation abgleichen und nachführen:** SRS, Architektur- und Implementierungsdokumentation mit dem aktuellen Code und der Konfiguration vergleichen, Widersprüche benennen und die betroffenen Beschreibungen aktualisieren. Offene Anforderungen bleiben als offen erkennbar; sie werden nicht entfernt, nur weil ihre Umsetzung fehlt.

Beispiel ohne vorgegebene Implementierung: Ein Skill „Projektstand übergeben“ fasst am Ende einer Sitzung Ziel, erledigte Arbeit, Entscheidungen, durchgeführte Prüfungen, offene Punkte und den nächsten sinnvollen Schritt in einer Projektstatusdatei zusammen. Er verweist auf vorhandene Dokumente, statt sie zu duplizieren. In der nächsten Sitzung lässt sich damit die Orientierung wiederherstellen; Repository und Git müssen mit der Notiz abgeglichen werden. Das Wiederherstellen des Kontexts autorisiert noch keine weiteren Änderungen. Wie Sie diesen Ablauf als Skill umsetzen und seine Ergebnisse überprüfen, entwickeln Sie selbst.

Die Beispiele sind Anregungen; eigene, zum Projekt passende Ideen sind ausdrücklich möglich, etwa eine Prüfung des Modbus-Registerplans oder des Nachrichtenvertrags. Zeigen Sie für **jeden Skill mindestens einen konkreten Aufruf** samt Ergebnis und Ihrer Prüfung dieses Ergebnisses. Ein bloß kopierter allgemeiner Prompt ohne erkennbaren Projektbezug genügt nicht. Übernommene Skills müssen Sie anpassen, verstehen und mit Quelle sowie Lizenz dokumentieren. Eine Übersetzung allein belegt noch keinen eigenen fachlichen Beitrag. Die Muster `savestate`, `commitstate` und `restorestate` sind mögliche Anwendungsfälle, aber keine vorgeschriebene Kombination. Ein Skill darf Commit und Push unterstützen, wenn sein ausdrücklicher Aufruf diese Aktionen einschließlich Ziel-Remote und Branch autorisiert; ein bloßer Journal-, Übergabe- oder Prüfauftrag löst keinen Push aus.

Mindestanforderungen

- Die eigenen Anwendungen verwenden weiterhin .NET 10 und lassen sich gemeinsam über die Solution bauen.
- Alle verpflichtenden Gerätesimulatoren liefern ihre über Modbus gelesenen Daten über RabbitMQ an den Viewer.
- Die vorhandene virtuelle Zeitsteuerung, Wettersimulation und Bedienung aus Aufgabe 02 bleiben erhalten.
- Die Regeln aus Aufgabe 02 zu `async`/`await`, `CancellationToken`, kontrolliertem Beenden und begründeten synchronen Bibliotheksgrenzen gelten weiter.
- Fehler werden sichtbar gemeldet. Ungültige oder veraltete Werte dürfen nicht stillschweigend als aktuelle, gültige Messwerte erscheinen.
- Broker-Ausfall oder Verbindungsabbruch führen nicht zu einer stillen Endlosschleife oder einem unkontrollierten Programmabbruch. Dokumentieren Sie das Wiederanlaufverhalten und den Umgang mit nicht übertragenen Daten.
- Zugangsdaten und lokale Konfigurationsdateien mit Geheimnissen werden nicht versioniert.
- Zentrales README und die README-Dateien der betroffenen Projekte beschreiben den aktuellen Stand, die Konfiguration, Startreihenfolge und bekannte Einschränkungen.

Nachweis der Funktion

Dokumentieren Sie folgenden Ablauf mit geeigneten Screenshots oder kurzen Konsolenprotokollen:

1. RabbitMQ wird aus der versionierten Compose-Konfiguration gestartet.
2. TimeSim, WeatherSim und alle verpflichtenden Gerätesimulatoren werden gestartet.
3. Gateway und Viewer werden gestartet; Nachrichten aller verpflichtenden Gerätesimulatoren sind erkennbar.
4. Ein Gerätezustand oder Sollwert wird verändert. Die Auswirkung ist im Simulator, in den gelesenen Modbus-Daten und in einer empfangenen Nachricht nachvollziehbar.

5. Die Simulation wird pausiert und fortgesetzt. Zeigen Sie, dass Messwerte weiterhin mit der richtigen virtuellen Zeit zugeordnet werden; dokumentieren Sie, ob während der Pause erneut veröffentlicht wird.
6. Die Verbindung zu einem Gerät und anschließend zum Broker wird unterbrochen und wiederhergestellt. Zeigen und erläutern Sie Fehleranzeige und Wiederanlauf.
7. Alle Anwendungen und der Broker werden kontrolliert beendet.

Führen Sie Ihre bisherigen automatisierten Tests weiter aus. Dokumentieren Sie außerdem den praktischen Einsatz des eigenen Skills jeder Person und den Abgleich der erreichten Ergebnisse mit den Akzeptanzkriterien Ihrer SRS.

Optionale Erweiterungen

Sie können zusätzlich Wetterdaten oder Zustandsinformationen von TimeSim über RabbitMQ veröffentlichen. Dokumentieren Sie hierfür einen passenden Nachrichtenvertrag. Wetterdaten können in späteren Ausbaustufen in InfluxDB mitgeführt werden.

Eine Speicherung in InfluxDB, REST-Schnittstellen und Grafana sind nicht Bestandteil dieser Aufgabe. Konzentrieren Sie sich zunächst auf einen verständlichen, funktionierenden Datenweg.

Abgabe

Aufgabe 03 ist eine **Teamaufgabe**. Führen Sie das gemeinsame Repository weiter.

- Die Abgabe erfolgt über git,
- mit annotiertem Git-Tag `lb03-2026`,
- vollständigem Commit-Hash dieses Tags.

Das Repository enthält:

- vollständigen Quellcode und die gemeinsam baubare .NET-10-Solution,
- Compose-Datei, benötigte Broker-Konfigurationen und Beispielkonfiguration ohne Geheimnisse,
- zentrales README zur Solution mit Dokumentation zur Architektur, Startreihenfolge, Bedienung
- README-Dateien zu den Projekten zur Bedienung, Steuerung
- Docker Konfiguration (Docker-Compose)
- aktualisierte SRS mit Akzeptanzkriterien,
- Nachrichtenvertrag, Routing-Dokumentation und mindestens eine Beispielnachricht,
- reproduzierbare Anleitung für etwaige verwendete externe Werkzeuge,
- eine aktuelle `AGENTS.md` oder `CLAUDE.md` für das Team,
- mindestens einen eigenen Skill pro Person mit eindeutiger Zuordnung und jeweiligem Anwendungsnachweis,
- Funktionsnachweise und dokumentierte Testergebnisse,
- die fortgeführten persönlichen Entwicklungsjournale.

Entwicklungsjournal

Führen Sie Ihr persönliches Entwicklungsjournal unter `docs/development-journal/<vorname-nachname>.md` weiter. Halten Sie wie bisher Arbeitsschritte, KI-Einsatz und entstandene Ergebnisse fest. Machen Sie insbesondere nachvollziehbar, wie Sie Planung und Implementierung getrennt und die KI-Ergebnisse geprüft haben.

Der Abgabetermin wird im Moodle-Kurs bekanntgegeben. Der Git-Stand zum angegebenen Tag ist der maßgebliche Abgabestand; spätere Verbesserungen verändern diesen Stand nicht.

Bewertungskriterien

Die folgende Checkliste macht transparent, was als substanzielle und nachvollziehbare Abgabe gilt:

- **Durchgängiger Datenweg:** Alle verpflichtenden Gerätesimulatoren liefern nachvollziehbare Daten über Modbus und RabbitMQ an den Viewer.
- **Nachrichtenvertrag:** Gerätekennung, virtuelle Zeit, Sequenznummer, Qualitätsstatus und Messwerte sind dokumentiert und korrekt zugeordnet.
- **Docker und Konfiguration:** RabbitMQ lässt sich anhand der versionierten Konfiguration und Anleitung reproduzierbar starten und stoppen.
- **Architektur und Robustheit:** Zuständigkeiten, Assemblies und Kommunikation sind verständlich getrennt; Fehler und Wiederanlauf werden nachvollziehbar behandelt.
- **KI-Arbeitsweise:** SRS, überprüfbare Umsetzungsschritte, eine erklärte `AGENTS.md` oder `CLAUDE.md` des Teams und ein tatsächlich verwendeter eigener Skill pro Person unterstützen die Entwicklung.
- **Nachvollziehbarkeit:** Dokumentation, Funktionsnachweise, Testergebnisse, Entwicklungsjournale und Git-Abgabestand sind vollständig.
- **Team- und Eigenleistung:** Jede Person kann ihre beiden verantworteten Module, den eigenen Beitrag und den gesamten Datenweg erklären.

Die Beurteilung stützt sich auf das lauffähige Gesamtsystem, Repository, Dokumentation und bei Bedarf eine kurze fachliche Rückfrage. Die optionale RabbitMQ-Anbindung von TimeSim und WeatherSim ist keine Voraussetzung für eine vollständige Pflichtabgabe.

Worauf achten wir?

Im Vordergrund steht ein funktionierender und verstandener Datenweg vom Gerätesimulator über Modbus und RabbitMQ bis zum Viewer. Ebenso wichtig sind eine eindeutige Zuordnung der Messwerte zur virtuellen Zeit, eine reproduzierbare Docker-Konfiguration und ein kontrollierter KI-Einsatz. Sie müssen Ihre Anforderungen, Architekturentscheidungen, den eigenen Code und die Ergebnisse Ihrer Prüfungen erklären können.